

Variant Types and Tail-Call Optimization

Learning Outcomes

by the **End** of the **Lecture**, **Students** that **Complete** the **Recommended Exercises** should be **Able** to:

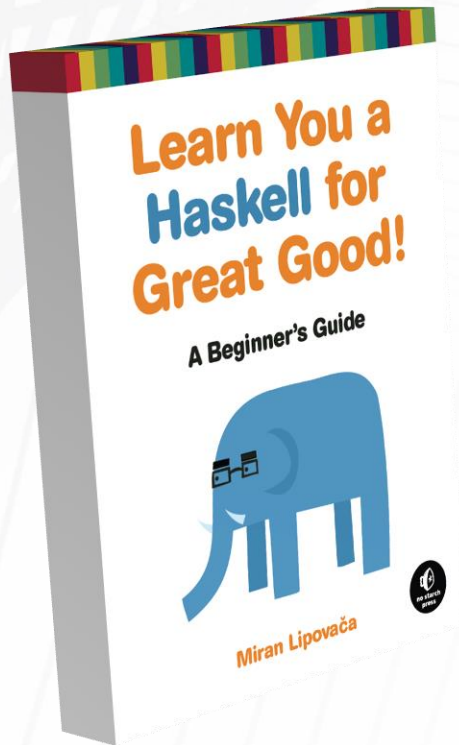
Explain Variant Record Types

Practice Pattern Matching (for Variant Record Types)

Trace the Evaluation of a Recursive Function

Use Tail-Call Optimization on Recursive Functions

Reading and Suggested Exercises



Read the following **Chapter(s)**:

None

Variant Record Types

in fact, **Lists** are actually a **Variant Record Type** with **Variant Record Types**, each **Variant** has a **Different Structure** (i.e., **Components**)

Any given List is Either:

the **Empty List**, for which `[]` is the **Constructor**, or

a **Cons Pair** `(h : t)`, for which `:` is the **Constructor**
(and where `h` is an Element and `t` is a List)

Variant Record Types

**to Support Variant Types, Language
Designers must Include Approaches for:**

**Constructing Each Variant on the data type
"Constructors"**

**Discriminating to which Variant an Instance belongs
"Variant Discrimination"**

**Extracting the Components of each Variant
"Component Extraction"**

Variant Record Types

Traditional Approaches Support each Task by:

Constructor Operators (for Constructors)

Typing Predicates (for Variant Discrimination)

Accessor Functions (for Component Extraction)

this **Approach** is **Reflected** in **Haskell** with the
Operator and Function set { `[]`, `:`, `head`, `tail`, `!!`, etc. }

n.b., the `!!` operator is actually be used for indexing into lists

Variant Record Types

the **Traditional Approach** can be **Considered Weaker** than the **Alternative**

How would you **Write** a **Function** that **Returns** the **5th Element** of a **list** of integers **If** the **Length** of the list is **At Least 5**, the **3rd Element** of a list **If** the **Length** of the list is **At Least 3**, and the value **-1 Otherwise?**

Variant Record Types

the **Traditional Approach** can be **Considered Weaker** than the **Alternative**

the **Traditional Approach** can be **Considered Weaker** than the **Alternative**

```
foo :: [Int] -> Int
foo x
  | length x >= 5      =
    head(tail(tail(tail(tail(x)))))
  | length x >= 3      =
    head(tail(tail(x)))
  | otherwise          = -1
```

the **Traditional Approach** can be **Considered Weaker** than the **Alternative**

```
foo :: [Int] -> Int
foo x
  | length x >= 5      = x !! 4
  | length x >= 3      = x !! 2
  | otherwise          = -1
```

Variant Record Types

the **Alternative** is to **Use Pattern Matching**
(instead of the **Operators** and **Functions**)

```
foo :: [Int] -> Int
foo ( _:_:_:_:x:_ )      = x
foo ( _:_:x:_ )          = x
foo x = -1
```

n.b., remember that each of the patterns is checked separately, from top to bottom

Additional Pattern Matching Exercises

**Pattern Matching is More Foundational than
Accessors Function and Typing Predicates**

**Using Only Pattern Matching,
How would you Write the Following?**

Accessor Functions: `head`, `tail`, `getNth` (i.e., `!!`)

Typing Predicate: `isEmpty`

Additional Pattern Matching Exercises

```
head :: [a] -> a  
head ['a', 'b', 'c', 'd'] = 'a'
```

```
tail :: [a] -> [a]  
tail ['a', 'b', 'c', 'd'] = ['b', 'c', 'd']
```

```
isEmpty :: [a] -> Bool  
isEmpty ['a', 'b', 'c', 'd'] = False
```

```
getNth :: Int -> [a] -> a  
getNth 2 ['a', 'b', 'c', 'd'] = 'c'
```


Additional Pattern Matching Exercises

Was `getNth` an Example of Primitive Recursion?

Why or Why Not?

Recursive Function Optimization

Consider this Recursive Implementation of Sum

```
sum :: [Int] -> Int
sum [] = 0
sum (h:t) = h + sum t
```

the **Addition** operation is **Performed After**
the **Return** of the **Recursive Call**

this **Entails** that a **Frame** for **Each Incomplete Call** be
Pushed onto the **Stack** to **Preserve** the **Context**

Recursive Function Optimization

Trace the Execution of `sum [1, 2, 4, 8]`

```
sum [] = 0  
sum (h:t) = h + sum t
```

Recursive Function Optimization

Trace the Execution of `sum [1, 2, 4, 8]`

```
sum [] = 0  
sum (h:t) = h + sum t
```

```
sum [1, 2, 4, 8]  
~ 1 + sum [2, 4, 8]  
~ 1 + (2 + sum [4, 8])  
~ 1 + (2 + (4 + sum [8]))  
~ 1 + (2 + (4 + (8 + sum [])))  
~ 1 + (2 + (4 + (8 + (0))))  
...
```



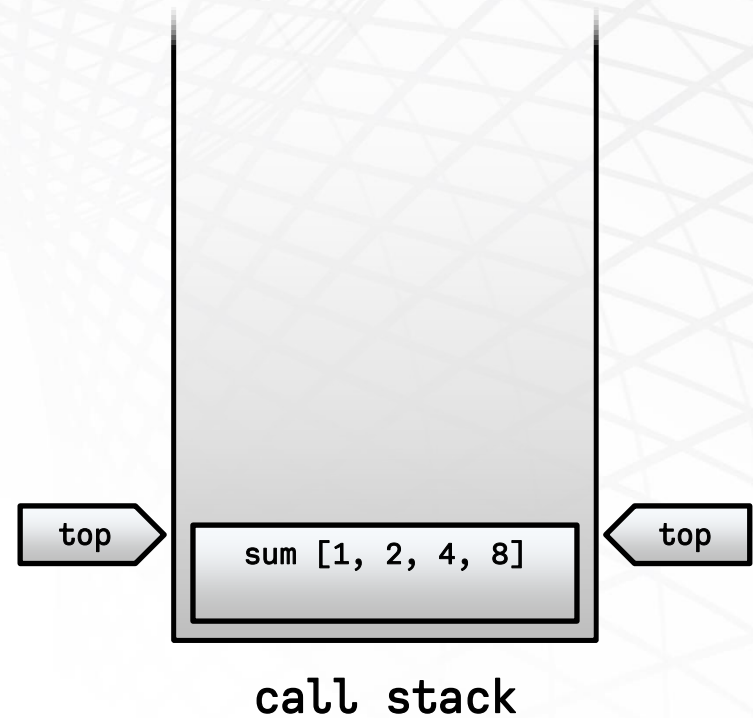
call stack

Recursive Function Optimization

Trace the Execution of `sum [1, 2, 4, 8]`

```
sum [] = 0  
sum (h:t) = h + sum t
```

```
sum [1, 2, 4, 8]  
~ 1 + sum [2, 4, 8]  
~ 1 + (2 + sum [4, 8])  
~ 1 + (2 + (4 + sum [8]))  
~ 1 + (2 + (4 + (8 + sum [])))  
~ 1 + (2 + (4 + (8 + (0))))  
...
```

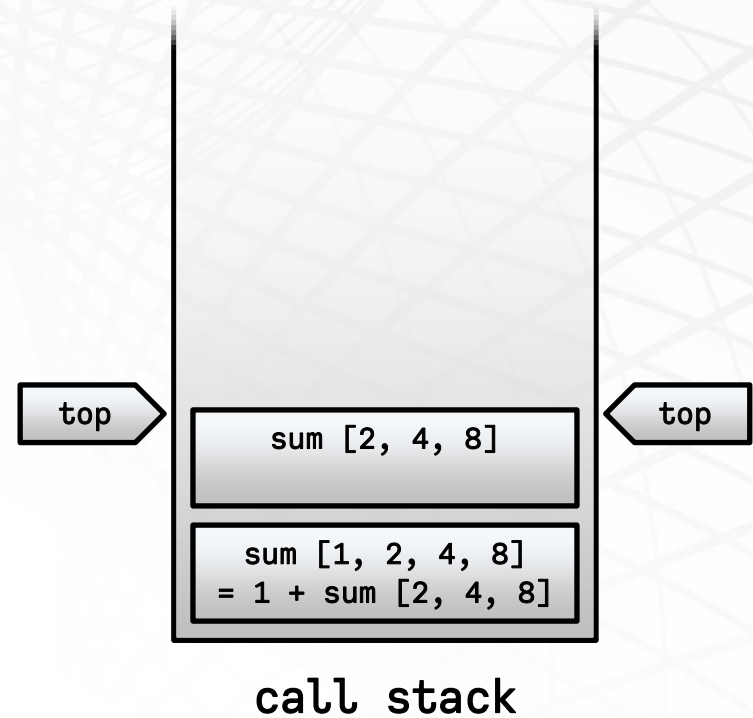


Recursive Function Optimization

Trace the Execution of `sum [1, 2, 4, 8]`

```
sum [] = 0  
sum (h:t) = h + sum t
```

```
sum [1, 2, 4, 8]  
~ 1 + sum [2, 4, 8]  
~ 1 + (2 + sum [4, 8])  
~ 1 + (2 + (4 + sum [8]))  
~ 1 + (2 + (4 + (8 + sum [])))  
~ 1 + (2 + (4 + (8 + (0))))  
...
```

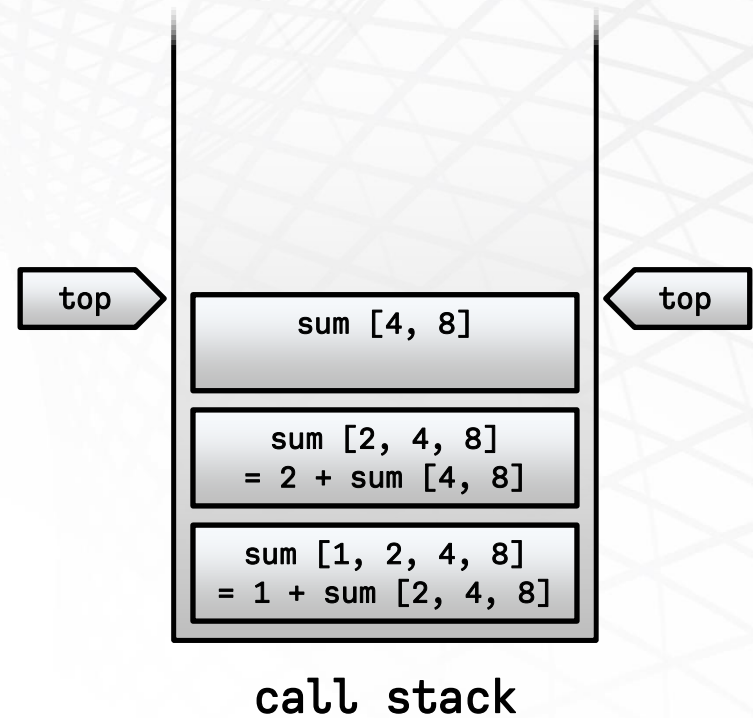


Recursive Function Optimization

Trace the Execution of `sum [1, 2, 4, 8]`

```
sum [] = 0  
sum (h:t) = h + sum t
```

```
sum [1, 2, 4, 8]  
~ 1 + sum [2, 4, 8]  
~ 1 + (2 + sum [4, 8])  
~ 1 + (2 + (4 + sum [8]))  
~ 1 + (2 + (4 + (8 + sum [])))  
~ 1 + (2 + (4 + (8 + (0))))  
...
```

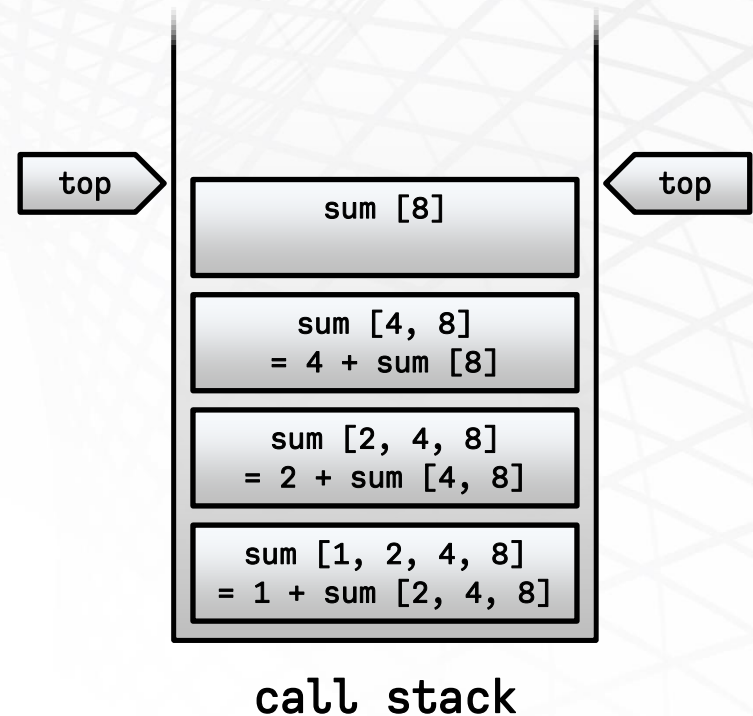


Recursive Function Optimization

Trace the Execution of `sum [1, 2, 4, 8]`

```
sum [] = 0  
sum (h:t) = h + sum t
```

```
sum [1, 2, 4, 8]  
~ 1 + sum [2, 4, 8]  
~ 1 + (2 + sum [4, 8])  
~ 1 + (2 + (4 + sum [8]))  
~ 1 + (2 + (4 + (8 + sum [])))  
~ 1 + (2 + (4 + (8 + (0))))  
...
```

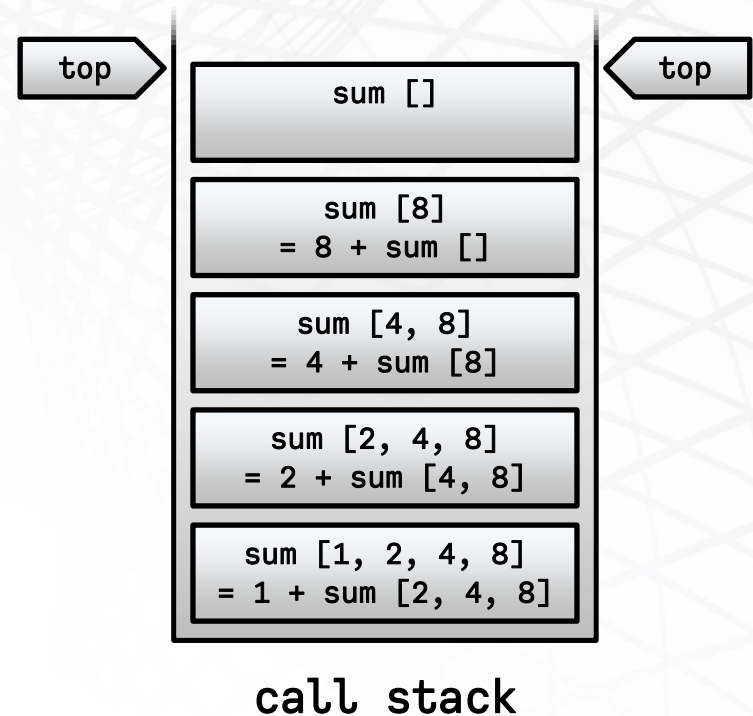


Recursive Function Optimization

Trace the Execution of `sum [1, 2, 4, 8]`

`sum [] = 0`
`sum (h:t) = h + sum t`

`sum [1, 2, 4, 8]`
~ `1 + sum [2, 4, 8]`
~ `1 + (2 + sum [4, 8])`
~ `1 + (2 + (4 + sum [8]))`
~ `1 + (2 + (4 + (8 + sum [])))`
~ `1 + (2 + (4 + (8 + (0))))`
...

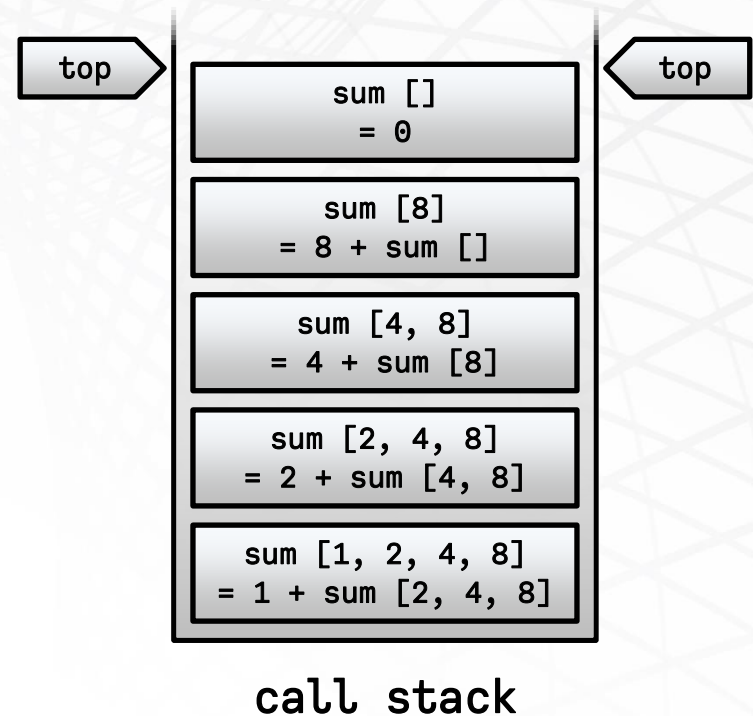


Recursive Function Optimization

Trace the Execution of `sum [1, 2, 4, 8]`

`sum [] = 0`
`sum (h:t) = h + sum t`

`sum [1, 2, 4, 8]`
~ `1 + sum [2, 4, 8]`
~ `1 + (2 + sum [4, 8])`
~ `1 + (2 + (4 + sum [8]))`
~ `1 + (2 + (4 + (8 + sum [])))`
~ `1 + (2 + (4 + (8 + (0))))`
...

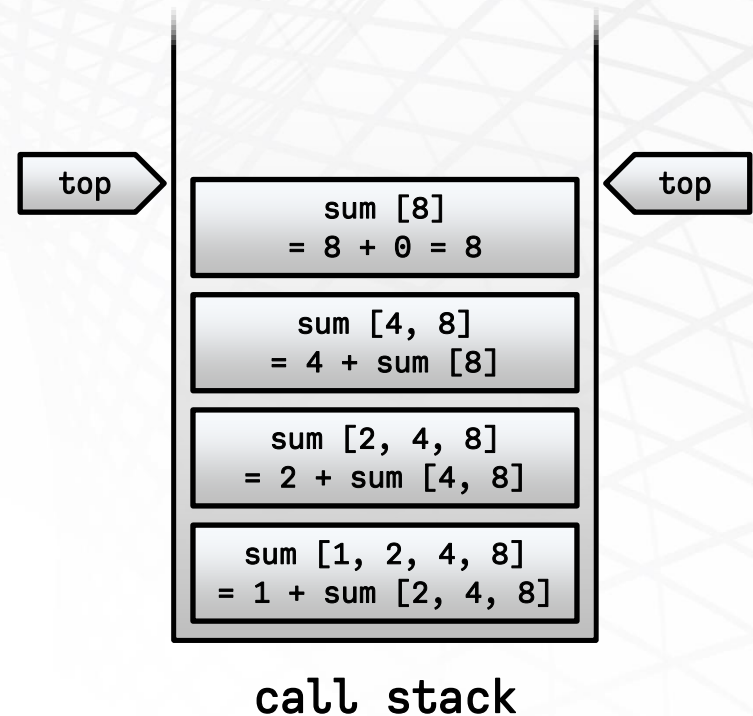


Recursive Function Optimization

Trace the Execution of `sum [1, 2, 4, 8]`

```
sum [] = 0  
sum (h:t) = h + sum t
```

```
sum [1, 2, 4, 8]  
~ 1 + sum [2, 4, 8]  
~ 1 + (2 + sum [4, 8])  
~ 1 + (2 + (4 + sum [8]))  
~ 1 + (2 + (4 + (8 + sum [])))  
~ 1 + (2 + (4 + (8 + (0))))  
...
```

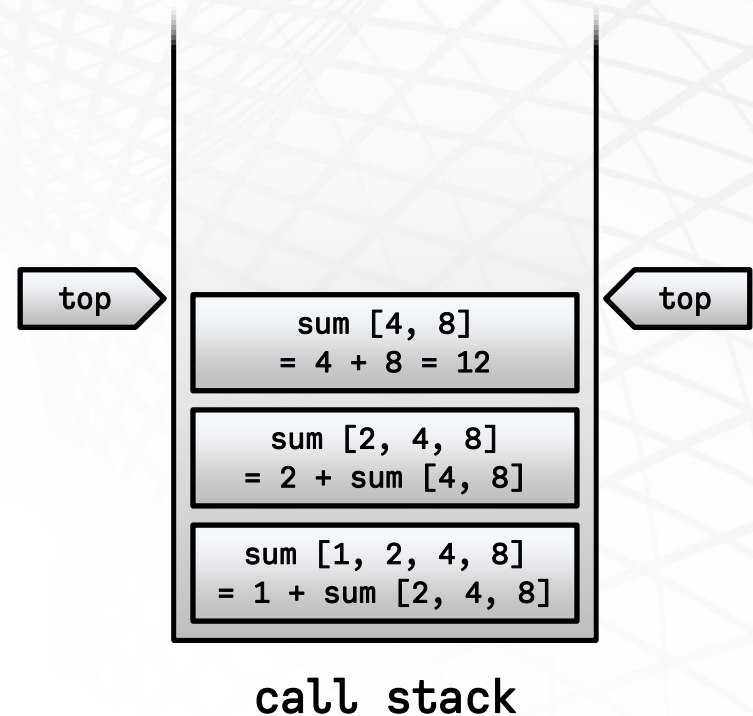


Recursive Function Optimization

Trace the Execution of `sum [1, 2, 4, 8]`

```
sum [] = 0  
sum (h:t) = h + sum t
```

```
sum [1, 2, 4, 8]  
~ 1 + sum [2, 4, 8]  
~ 1 + (2 + sum [4, 8])  
~ 1 + (2 + (4 + sum [8]))  
~ 1 + (2 + (4 + (8 + sum [])))  
~ 1 + (2 + (4 + (8 + (0))))  
...
```

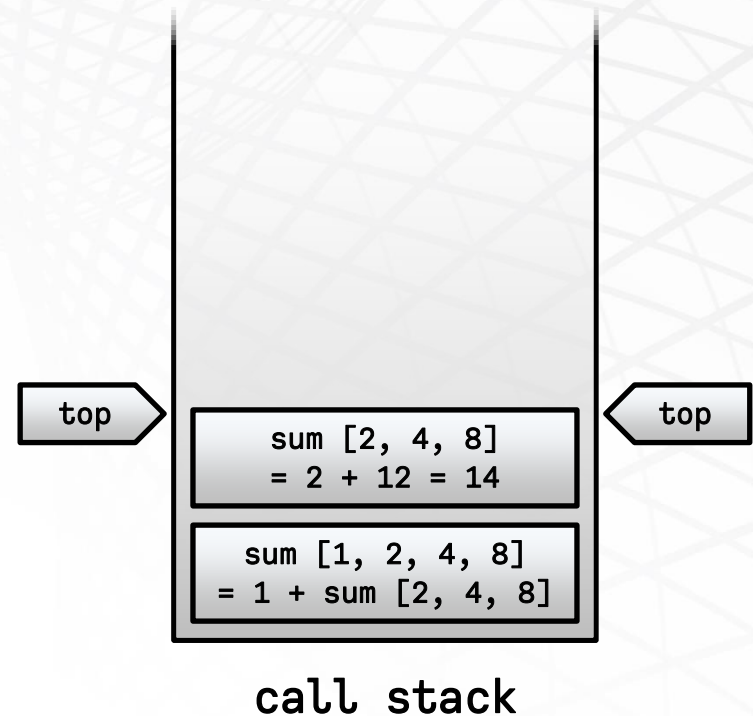


Recursive Function Optimization

Trace the Execution of `sum [1, 2, 4, 8]`

```
sum [] = 0  
sum (h:t) = h + sum t
```

```
sum [1, 2, 4, 8]  
~ 1 + sum [2, 4, 8]  
~ 1 + (2 + sum [4, 8])  
~ 1 + (2 + (4 + sum [8]))  
~ 1 + (2 + (4 + (8 + sum [])))  
~ 1 + (2 + (4 + (8 + (0))))  
...
```

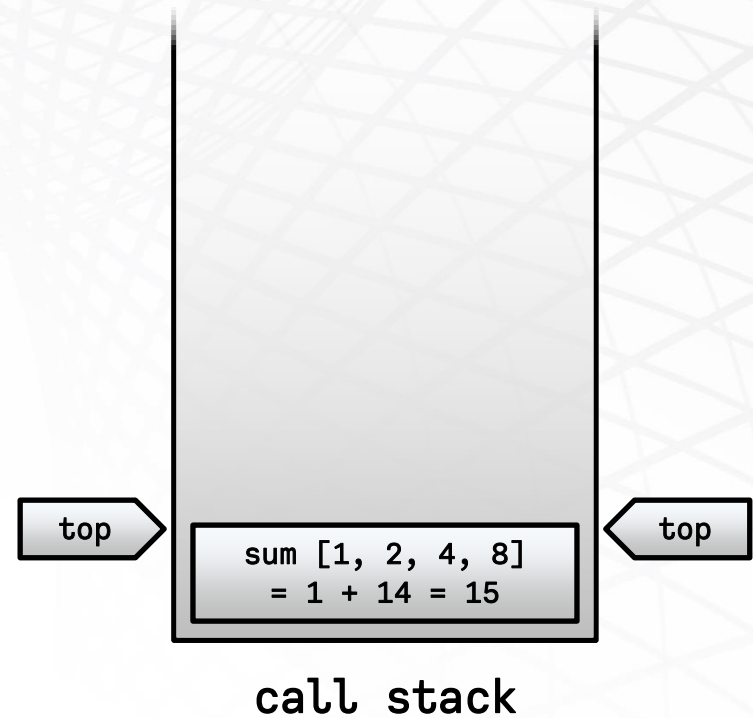


Recursive Function Optimization

Trace the Execution of `sum [1, 2, 4, 8]`

```
sum [] = 0  
sum (h:t) = h + sum t
```

```
sum [1, 2, 4, 8]  
~ 1 + sum [2, 4, 8]  
~ 1 + (2 + sum [4, 8])  
~ 1 + (2 + (4 + sum [8]))  
~ 1 + (2 + (4 + (8 + sum [])))  
~ 1 + (2 + (4 + (8 + (0))))  
...
```



Recursive Function Optimization

Trace the Execution of `sum [1, 2, 4, 8]`

`sum [] = 0`

`sum (h:t) = h + sum t`

`sum [1, 2, 4, 8]`

`~ 1 + sum [2, 4, 8]`

`~ 1 + (2 + sum [4, 8])`

`~ 1 + (2 + (4 + sum [8]))`

`~ 1 + (2 + (4 + (8 + sum [])))`

`~ 1 + (2 + (4 + (8 + (0))))`

15



call stack

Recursive Function Optimization

the **Addition** $1 + \text{sum}[2, 4, 8]$ had to **Wait**
until the **Result** of $\text{sum}[2, 4, 8]$ was **Computed**...

...but the **Addition** $2 + \text{sum}[4, 8]$ had to **Wait**
until the **Result** of $\text{sum}[4, 8]$ was **Computed**...

...which had to **Wait** ... etc.

Recursion can Entail a Great Deal of Waiting...
...but **Was the Waiting Actually Necessary?**

Recursive Function Optimization

Does this Addition...

$$\begin{array}{ccccccc} & \underbrace{\hspace{1.5cm}} & & & & & \\ 1 & + & 2 & + & 4 & + & 8 \\ & & \underbrace{\hspace{1.5cm}} & & & & \end{array}$$

...Really Need to Wait for this Result?

Operations that Use the Return Value of the Recursive Call as an Argument Need Not necessarily Wait ...

...as long as the Partial Result can be Passed Forward

Recursive Function Optimization

Consider this **Alternative** (but still **Recursive**) **Implementation of Sum**

```
sum :: [Int] -> Int  
sum x = sum' x 0
```

```
sum' :: [Integer] -> Integer -> Integer  
sum' [] a = a  
sum' (h:t) a = sum' t (h+a)
```

Trace the Execution of `sum [1, 2, 4, 8]`

Recursive Function Optimization

```
sum x = sum' x 0
sum' [] a = a
sum' (h:t) a = sum' t (h+a)
```

```
sum [1, 2, 4, 8]
~ sum' [1, 2, 4, 8] 0
~ sum' [2, 4, 8] (1 + 0)
~ sum' [2, 4, 8] 1
~ sum' [4, 8] (2 + 1)
~ sum' [4, 8] 3
~ sum' [8] (4 + 3)
~ sum' [8] 7
~ sum' [] (8 + 7)
~ sum' [] 15
~ 15
```

Recursive Function Optimization

the **Recursive Call** to `sum'` is a **Tail Call**

```
sum' :: [Integer] -> Integer -> Integer
sum' [] a = a
sum' (h:t) a = sum' t (h+a)
```

the **Return Value** of a **Tail Call** can be
Immediately Returned by the **Calling Function**

there is **No** actual **Need** to **Push** a **Stack Frame**
as there is **No Work Waiting** for a **Return**

Recursive Function Optimization

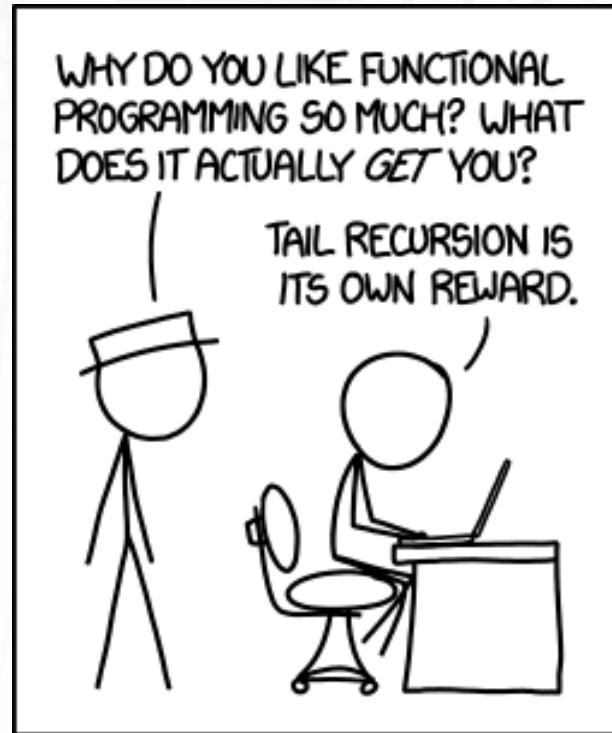
the **Parameter** `a` is known as an **Accumulator**

```
sum' :: [Integer] -> Integer -> Integer
      sum' [] a = a
      sum' (h:t) a = sum' t (h+a)
```

Using an Accumulator for Tail Call Optimization
can result in a **Performance Improvement**

some **Compilers** can even **Recognize Opportunities** for
Tail Calls and perform the **Optimization Themselves**

"Functional" by Randall Munroe



Functional programming combines the flexibility and power of abstract mathematics with the intuitive clarity of abstract mathematics.

<https://xkcd.com/1270/>

Recursive Function Optimization

How would you **Optimize** the **Factorial Function**
using a **Tail Call**?

```
fact :: Integer -> Integer
fact 0 = 1
fact n = n * (fact (n-1))
```

Recursive Function Optimization

How would you **Perform Tail Call Optimization**
on each of these **Recursive Design Patterns**?

```
foldOper [] = identity  
foldOper (h:t) = h oper foldOper(t)
```

```
mapFunc [] = []  
mapFunc (h:t) = (func h) : (mapFunc t)
```

```
filterPred [] = []  
filterPred (h:t)  
  | (pred h) == True = h : (filterPred t)  
  | otherwise = (filterPred t)
```